

```
In [ ]: import marimo as mo
```

Trabalho Prático: Sudoku Genérico como CSP

Contexto

O Sudoku clássico — uma grelha $n^2 \times n^2$ onde cada linha, cada coluna e cada bloco $n \times n$ tem de conter todos os valores de 1 a n^2 sem repetições — é um exemplo canónico de **problema de satisfação de restrições (CSP)**: a "regra" é sempre a mesma (um conjunto de células tem de ter valores todos diferentes), o que muda de linha para linha, de coluna para coluna e de bloco para bloco é apenas **que células pertencem a esse conjunto**.

Isso sugere uma abstração única — um grupo de células com a restrição "todos diferentes", opcionalmente com algumas células já fixas a um valor — a partir da qual linhas, colunas, blocos e ainda outras variantes de Sudoku (diagonais, regiões irregulares, grelhas sobrepostas, etc.) podem ser todas construídas sem duplicar lógica de restrição nenhuma.

Este é um problema de **modelação e resolução de CSP**. Cabe-te a ti escolher a técnica de resolução e justificá-la — o enunciado não fornece código de modelação nem de apresentação de resultados, apenas a interface que o teu notebook tem de expor (secção seguinte) para poder ser testado automaticamente.

Objetivo

Construir, num notebook Marimo, um gerador/resolvedor de Sudoku $n^2 \times n^2$ (com n parametrizável, tipicamente $n = 3$) que:

1. representa qualquer **grupo de células com restrição "todos diferentes"** através de uma classe genérica (secção "`box`" — grupo genérico de células),
2. constrói **linhas, colunas e blocos** como casos particulares dessa classe genérica — os blocos através de uma especialização dedicada a blocos $n \times n$, as linhas e colunas através de uma especialização dedicada a sequências retas de células (secção "`cube`" e "`path`"),
3. gera **aleatoriamente** um subconjunto de células já preenchidas (as "pistas" iniciais do puzzle), usando a mesma abstração genérica (secção "Geração aleatória de pistas"),
4. monta o modelo completo (linhas + colunas + blocos + pistas) e o resolve como CSP, devolvendo a grelha preenchida ou sinalizando que não há solução (secção "Resolução").

Requisitos obrigatórios

O teu notebook tem de expor, com este comportamento, os seguintes elementos (os nomes propostos abaixo são sugestões que facilitam a correção automática — podes usar outros, desde que documentes a correspondência):

`box` — grupo genérico de células (R1)

Uma classe que representa **qualquer** conjunto de células da grelha às quais se aplica a restrição "todos os valores diferentes", com algumas delas possivelmente já fixas:

- guarda internamente uma associação `(linha, coluna) → valor` ou `None` (`None` = célula livre; um inteiro = célula fixa/pinada a esse valor);
- um construtor que aceita opcionalmente esse conjunto inicial de células (vazio por omissão);
- um método `add(i, j, val=None)` que acrescenta a célula `(i, j)` ao grupo, opcionalmente fixando-a a `val`, e que **rejeita** (levanta exceção) coordenadas fora da grelha ou valores fora do intervalo $[1, n^2]$;
- uma forma de obter a representação do grupo como matriz $n^2 \times n^2$, com zeros nas células não pertencentes ao grupo ou não fixas, e o valor fixo nas restantes.

Esta classe **não deve saber nada** sobre linhas, colunas, blocos ou Sudoku — só sabe lidar com "um conjunto de células, algumas fixas". Essa generalidade é o que te vai permitir, mais tarde, tratar da mesma forma linhas, colunas, blocos, pistas aleatórias e (nas extensões opcionais) diagonais ou regiões irregulares.

`cube` e `path` — duas formas concretas de grupo (R2, R3)

A partir da classe genérica, define duas especializações:

- **R2.** Um grupo que representa o **bloco** $n \times n$ cujo canto superior esquerdo é a célula $(i \cdot n, j \cdot n)$, parametrizado pelos índices de bloco (i, j) com $0 \leq i, j < n$.
- **R3.** Um grupo que representa o **troço reto** (horizontal ou vertical) de células entre duas coordenadas `inicio` e `fim`, inclusive — tem de funcionar tanto para `fim` "depois" de `inicio` como "antes" (ou seja, percorrer a sequência em qualquer sentido).

Geração aleatória de pistas (R4)

Uma função que devolve um grupo (`box`) com k células escolhidas aleatoriamente na grelha, cada uma fixa a um valor também escolhido aleatoriamente em $[1, n^2]$ (k deve ter um valor por omissão razoável, por exemplo da ordem de n). Repara que esta função **não precisa de nenhuma classe nova** — o resultado é, de novo, apenas um `box`.

Modelo e resolução (R5, R6)

- **R5.** Um modelo de CSP para a grelha $n^2 \times n^2$, com uma variável inteira por célula, cada uma no intervalo $[1, n^2]$; um método que recebe **um número arbitrário de grupos** (`box` , `cube` , `path` , ou pistas aleatórias — o modelo não deve distinguir a sua origem) e, para cada um, impõe que as suas células sejam todas diferentes e fixa as que tiverem valor atribuído; e um método de resolução que devolve a grelha preenchida ou sinaliza, de forma distinguível, que o puzzle não tem solução.
- **R6.** Um Sudoku $n^2 \times n^2$ completo é montado juntando: todas as linhas, todas as colunas, todos os blocos $n \times n$ e (pelo menos) um grupo de pistas aleatórias — e resolvido.

Como testar/validar

O teu notebook (ou um ficheiro de testes à parte) tem de verificar automaticamente, para uma grelha resolvida:

- que cada linha, cada coluna e cada bloco $n \times n$ contém exatamente os valores $1 \dots n^2$, sem repetições;
- que as células fixadas pelas pistas aleatórias mantêm, na solução, o valor com que foram fixadas;
- que `add` (ou equivalente) rejeita coordenadas fora da grelha e valores fora de $[1, n^2]$.

Corre o fluxo completo (gerar pistas aleatórias → montar linhas + colunas + blocos + pistas → resolver → validar) pelo menos uma vez com $n = 3$ (Sudoku clássico 9×9) e confirma que também funciona com outro valor de n (ex.: $n = 2$, grelha 4×4), para garantir que nada está fixo a 9×9 no teu código.

O que é deixado ao teu critério

O enunciado define **que abstrações** o notebook tem de expor e **que comportamento** têm de ter, não **como** as deves implementar. Ficam ao teu critério, desde que justificadas no notebook:

- a técnica e biblioteca de resolução do CSP (CP-SAT do OR-Tools é a sugestão da disciplina, mas és livre de escolher outra abordagem de Lógica Computacional, justificando a escolha);
- a estrutura de dados interna do grupo genérico (dicionário, matriz esparsa, etc.);
- a forma de apresentar a grelha resultante (texto, tabela, `mo.ui` , gráfico — o que achares mais claro);
- o comportamento exato quando o puzzle gerado aleatoriamente não tem solução (podes, por exemplo, tentar novas pistas aleatórias até obteres um puzzle solúvel, ou simplesmente reportar o insucesso — justifica a escolha).

Extensões opcionais (bónus)

A generalidade do `box` é o que torna estas extensões possíveis sem tocar no modelo CSP em si — cada uma acrescenta apenas **novos grupos** de células:

- **Sudoku diagonal (X-Sudoku)**: acrescenta um grupo (`box` , sem precisar de nova subclasse) para cada uma das duas diagonais principais, também elas restritas a "todos diferentes".
- **Sudoku irregular (jigsaw)**: substitui os blocos $n \times n$ regulares por regiões de forma arbitrária mas do mesmo tamanho, cada uma representada como um `box` construído célula a célula em vez de por `cube` .
- **Hyper-Sudoku / Windoku**: acrescenta 4 blocos extra (também `box` , de forma semelhante a `cube` mas sem estarem alinhados com a grelha $n \times n$ de blocos) sobrepostos aos existentes.
- **Escala**: mostra que o teu código funciona (talvez mais devagar) para $n = 6$ (grelha 36×36) sem alterações, e discute os limites de desempenho que encontraste.
- **Sudoku tridimensional** define a estrutura de "boxes" numa grelha $n^2 \times n^2 \times n^2$.

Porquê o CP-SAT? Usámos o CP-SAT do OR-Tools por ser o solver sugerido pela disciplina e porque se ajusta diretamente ao Sudoku: cada célula é uma variável inteira entre 1 e N, e a restrição "todos diferentes" (o `alldifferent` do Capítulo 2) já vem implementada (`add_all_different`), sem ser preciso escrevê-la à mão. Quando o puzzle não tem solução, o solver devolve um estado diferente de `OPTIMAL` e `FEASIBLE` , e o método `resolve` devolve `None` , o que permite distinguir esse caso de uma grelha preenchida.

Porquê uma só restrição? O modelo só precisa de conhecer "todos diferentes" porque linhas, colunas, blocos e células iniciais obedecem todos à mesma regra: as suas células têm de ter valores diferentes. O que muda de grupo para grupo é apenas *quais* células lhe pertencem, e isso está guardado no `box` . Assim, o método `adiciona` aplica a mesma regra a qualquer grupo, sem precisar de saber se é uma linha, uma coluna ou um bloco.

Porquê um dicionário no `box` . Guardámos as células do `box` num dicionário (linha, coluna) $\rightarrow$ valor ou `None` porque assim distinguimos três situações: se a chave não existe, a célula não pertence ao grupo; se existe com `None` , pertence mas está livre; se existe com um número, está fixa. Assim o modelo só guarda as células do grupo (uma linha tem 9 das 81) e a matriz $n^2 \times n^2$ só é construída quando é precisa, com `matriz()` .

Como se mostra a grelha. A grelha é mostrada em texto, uma linha por linha da grelha, com `|` entre blocos e traços entre filas de blocos, por ser simples e legível. A função `mostra` usa `n` e `N = n*n` em vez de valores fixos, por isso funciona para qualquer `n` , mas só alinha bem com números de um dígito (`n ≤ 3`).

Puzzle sem solução. Se não há solução, o `resolve` devolve `None` . Como as células iniciais são sorteadas sem olhar às regras, podem contradizer-se, e por isso

`sudoku_resolvido` sorteia de novo até 50 vezes, devolvendo `None, None` se esgotar o limite. Optámos por voltar a sortear, e não só por reportar o insucesso, porque os validadores precisam de uma grelha resolvida, e o limite evita um ciclo infinito.

R1: `box` . O `box` guarda um grupo de células num dicionário `(linha, coluna) → valor` ou `None` . O `add` rejeita com `ValueError` coordenadas fora da grelha e valores fora de `[1, N]`, e o método `matriz()` devolve o grupo como matriz $n^2 \times n^2$. O construtor aceita, opcionalmente, um conjunto inicial de células, que passa pelo `add` e é por isso validado.

```
In [ ]: class box:
    def __init__(self, n, celulas=None):
        self.n = n
        self.N = n * n
        self.celulas = {}
        if celulas is not None:
            for (i, j), val in celulas.items():
                self.add(i, j, val)

    def add(self, i, j, val=None):
        if not (0 <= i < self.N and 0 <= j < self.N):
            raise ValueError("coordenadas fora da grelha")
        if val is not None and (self.N < val or val < 1):
            raise ValueError("valor fora, insira um valor entre 1 e "+ str
(self.N) )

        self.celulas[(i, j)] = val

    def matriz(self):
        m = []
        for i in range(self.N):
            linha = []
            for j in range(self.N):
                # AQUI: ir buscar o valor de (i, j) ao dicionário,
                valor = self.celulas.get((i,j))
                if valor is None :
                    valor=0
                # e acrescentar 0 se não existir OU se for None
                linha.append(valor)
            m.append(linha)

        return m
```

```
In [ ]: b2 = box(2)
b2.add(0, 0, 3)
b2.add(1, 1)
print(b2.matriz())
```

```
[[3, 0, 0, 0], [0, 0, 0, 0], [0, 0, 0, 0], [0, 0, 0, 0]]
```

```
In [ ]: print(box(2, {(0, 0): 3, (1, 1): None}).celulas)

{(0, 0): 3, (1, 1): None}
```

R2: cube. O `cube(n, i, j)` herda do `box` e acrescenta as células do bloco (i, j), a partir do canto (i-n, j-n).

```
In [ ]: class cube(box):
        def __init__(self, n, i, j):
            super().__init__(n)
            for a in range(n):
                for b in range(n):
                    self.add(i*n + a, j*n + b)
```

```
In [ ]: c = cube(2, 1, 1)
        print(c.celulas)
```

```
{(2, 2): None, (2, 3): None, (3, 2): None, (3, 3): None}
```

```
In [ ]: print(len(cube(3, 1, 2).celulas))
```

```
9
```

```
In [ ]: def sinal(x):
        if (x<0):
            return (-1)
        elif (x>0):
            return (1)
        else:
            return (0)
```

```
In [ ]: print(sinal(5), sinal(-3), sinal(0))
```

```
1 -1 0
```

R3: path. O `path` herda do `box` e acrescenta as células do trecho reto entre `inicio` e `fim`, em qualquer sentido. Recusa treços que não sejam retos, o que é uma decisão nossa.

```
In [ ]: class path(box):

        def __init__(self, n, inicio, fim):
            super().__init__(n)
            if (fim[0] != inicio[0]) and (fim[1] != inicio[1]):
                raise ValueError("Nem as linhas nem as colunas sao
compativeis,movimento nao permitido")
            di = sinal(fim[0] - inicio[0])      # passo da linha
            dj = sinal(fim[1] - inicio[1])      # passo da coluna
            passos = max(abs(fim[0] - inicio[0]), abs(fim[1] - inicio[1]))
            for k in range(passos + 1):
                self.add( inicio[0]+di*k, inicio[1]+dj*k )
```

```
In [ ]: p2 = path(2, (0, 1), (3, 1))
        print(p2.celulas)
```

```
{(0, 1): None, (1, 1): None, (2, 1): None, (3, 1): None}
```

```
In [ ]: print(path(2, (2, 3), (2, 0)).celulas)
```

```
{(2, 3): None, (2, 2): None, (2, 1): None, (2, 0): None}
```

R4: células iniciais. No enunciado, "pistas" são as células já preenchidas no início. Mantivemos esse nome em `pistas` e `pistas_ok`, mas referem-se sempre às células iniciais. A função devolve um `box` com k células escolhidas ao acaso, sem repetir, cada uma com um valor aleatório em [1, N].

```
In [ ]: import random

def pistas(n, k=3):
    N = n * n
    b = box(n)
    todas = [(i, j) for i in range(N) for j in range(N)]
    escolhidas = random.sample(todas, k)
    for (i, j) in escolhidas:
        b.add(i, j, random.randint(1, N))
    return b
```

```
In [ ]: print(pistas(3).celulas)
print(len(pistas(2, 5).celulas))
```

```
{(0, 4): 3, (3, 6): 5, (7, 5): 2}
5
```

R5: modelo. A classe `Sudoku` cria uma variável inteira por célula. O método `adiciona` impõe "todos diferentes" a qualquer número de grupos e fixa as células com valor, e `resolve` devolve a grelha ou `None`.

```
In [ ]: from ortools.sat.python import cp_model

class Sudoku:
    def __init__(self, n):
        self.n = n
        self.N = n * n
        self.modelo = cp_model.CpModel()
        self.v = {}
        for i in range(self.N):
            for j in range(self.N):
                self.v[(i, j)] = self.modelo.new_int_var(1, self.N, f"v_{i}_{j}")

    def adiciona(self, *grupos):
        for g in grupos:
            vars_g = []
            for c in g.celulas:
                vars_g.append(self.v[c])
            self.modelo.add_all_different(vars_g)
            for (i, j), val in g.celulas.items():
                if val is not None:
                    self.modelo.add(self.v[(i, j)] == val)

    def resolve(self):
        solver = cp_model.CpSolver()
        estado = solver.solve(self.modelo)
        if estado == cp_model.OPTIMAL or estado == cp_model.FEASIBLE:
            grelha = []
```

```

        for i in range(self.N):
            linha = []
            for j in range(self.N):
                linha.append(solver.value(self.v[i, j]))
            grelha.append(linha)
        return grelha
    return None

```

```

In [ ]: g_imp = box(2)
        g_imp.add(0, 0, 3)
        g_imp.add(0, 1, 3)
        s_imp = Sudoku(2)
        s_imp.adiciona(g_imp)
        print(s_imp.resolve())      # deve dar None

```

None

```

In [ ]: def linhas(n):
        N = n * n
        resultado = []
        for i in range(N):
            resultado.append(path(n, (i, 0), (i, N - 1)))
        return resultado

```

```

In [ ]: def blocos(n):
        resultado = []
        for i in range(n):
            for j in range(n):
                resultado.append(cube(n,i,j))
        return resultado

```

```

In [ ]: def colunas(n):
        N = n * n
        resultado = []
        for j in range(N):
            resultado.append(path(n, (0, j), (N - 1, j)))
        return resultado

```

```

In [ ]: print(len(blocos(3)), len(blocos(3)[0].celulas))

```

9 9

```

In [ ]: print(len(linhas(3)), len(linhas(3)[0].celulas), len(colunas(3)), len(colunas(3)
        [0].celulas))

```

9 9 9 9

R6: Sudoku completo. A função `sudoku_completo` junta todas as linhas, colunas, blocos e as células iniciais, e resolve.

```

In [ ]: def sudoku_completo(n, k=3):
        s = Sudoku(n)
        s.adiciona(*linhas(n))
        s.adiciona(*colunas(n))      # as colunas

```

```
s.adiciona(*blocos(n))          # os blocos
p = pistas(n, k)
s.adiciona(p)
return p, s.resolve()
```

```
In [ ]: pist_r6, grel_r6 = sudoku_resolvido(3)
print(pist_r6.celulas)
print(grel_r6)
```

```
{(8, 5): 1, (4, 0): 8, (3, 7): 3}
[[4, 1, 9, 5, 8, 7, 3, 6, 2], [3, 6, 5, 4, 1, 2, 8, 7, 9], [2, 7, 8, 3, 9, 6, 1, 5, 4], [1, 5, 4, 2, 7, 9, 6, 3, 8], [8, 3, 2, 1, 6, 4, 7, 9, 5], [7, 9, 6, 8, 3, 5, 2, 4, 1], [5, 2, 1, 7, 4, 3, 9, 8, 6], [6, 4, 3, 9, 2, 8, 5, 1, 7], [9, 8, 7, 6, 5, 1, 4, 2, 3]]
```

Validação. Os validadores trabalham só sobre a grelha final, sem usar o modelo, para que um erro no modelo não passe despercebido. Cada um foi testado com grelhas válidas e com grelhas erradas de propósito, para mostrar que sabe dar `False`. `linhas_ok` verifica que cada linha, depois de ordenada, é igual a `[1, ..., N]`.

```
In [ ]: def linhas_ok(grelha, n):
        N = n * n
        esperado = list(range(1, N + 1))
        for linha in grelha:
            if sorted(linha) != esperado:
                return False
        return True
```

```
In [ ]: print(linhas_ok(grel_r6, 3))
```

True

```
In [ ]: print(linhas_ok([[1, 2, 3, 3], [1, 2, 3, 4], [1, 2, 3, 4], [1, 2, 3, 4]], 2))
```

False

`colunas_ok` reaproveita `linhas_ok` depois de virar a grelha com `transposta`, porque uma coluna é uma linha vista de lado.

```
In [ ]: def transposta(grelha):
        N = len(grelha)
        t = []
        for j in range(N):
            coluna = []
            for i in range(N):
                coluna.append(grelha[i][j])
            t.append(coluna)
        return t
```

```
In [ ]: print(transposta([[1, 2], [3, 4]]))
```

[[1, 3], [2, 4]]

```
In [ ]: def colunas_ok(grelha, n):  
        return linhas_ok(transposta(grelha), n)
```

```
In [ ]: print(colunas_ok([[1, 2, 3, 4], [1, 2, 3, 4], [1, 2, 3, 4], [1, 2, 3, 4]], 2))
```

False

```
In [ ]: print(colunas_ok(grel_r6, 3))
```

True

`blocos_ok` junta os valores de cada bloco e verifica o mesmo. O teste com o bloco de baixo à direita errado serve para provar que lê todos os blocos.

```
In [ ]: def blocos_ok(grelha, n):  
        N = n * n  
        esperado = list(range(1, N + 1))  
        for i in range(n):  
            for j in range(n):  
                bloco = []  
                for a in range(n):  
                    for b in range(n):  
                        bloco.append(grelha[i*n + a][j*n + b])  
                if sorted(bloco) != esperado:  
                    return False  
        return True
```

```
In [ ]: print(blocos_ok([[1, 2, 3, 4], [3, 4, 1, 2], [2, 1, 4, 3], [4, 3, 4, 3]], 2))  #  
        deve dar False
```

False

```
In [ ]: print(blocos_ok(grel_r6, 3))
```

True

```
In [ ]: print(blocos_ok([[1, 2, 3, 4], [2, 1, 4, 3], [3, 4, 1, 2], [4, 3, 2, 1]], 2))
```

False

`pistas_ok` verifica que as células iniciais mantêm o seu valor na solução.

```
In [ ]: def pistas_ok(grelha, pistas):  
        for (i, j), val in pistas.celulas.items():  
            if grelha[i][j] != val :  
                return False  
        return True
```

```
In [ ]: print(pistas_ok(grel_r6, pist_r6))
```

True

```
In [ ]: g_p = box(2)  
        g_p.add(0, 0, 2)  
        print(pistas_ok([[1, 2, 3, 4], [3, 4, 1, 2], [2, 1, 4, 3], [4, 3, 2, 1]], g_p))
```

False

```
In [ ]: def sudoku_resolvido(n, k=3, tentativas=50):
        for t in range(tentativas):
            p, g = sudoku_completo(n, k)
            if g is not None:
                return p, g
        return None, None
```

`valida_completo` corre o fluxo todo (gerar, montar, resolver e validar), e deu `True` com `n=3` e com `n=2`.

```
In [ ]: def valida_completo(n, k=3):
        p, g = sudoku_resolvido(n, k)
        if g is None:
            return False
        return linhas_ok(g, n) and colunas_ok(g, n) and blocos_ok(g, n) and
        pistas_ok(g, p)
```

```
In [ ]: print(valida_completo(3))
```

True

```
In [ ]: print(valida_completo(2))
```

True

`rejeita` confirma que o `add` recusa coordenadas e valores inválidos, sem deixar o notebook a vermelho.

```
In [ ]: def rejeita(n, i, j, val=None):
        b = box(n)
        try:
            b.add(i, j, val)
        except ValueError:
            return True
        return False
```

```
In [ ]: print(rejeita(3, 9, 0))      # Linha fora da grelha
        print(rejeita(3, -1, 0))     # Linha negativa
        print(rejeita(3, 0, 0, 10))  # valor acima de 9
        print(rejeita(3, 0, 0, 0))   # valor abaixo de 1
        print(rejeita(3, 0, 0, 5))   # tudo válido
```

True

True

True

True

False

```
In [ ]: def mostra(grelha, n):
        N = n * n
        for i in range(N):
            if i % n == 0 and i != 0:
                print("-" * (2*N + 2*n - 3))
```

```

partes = []
for j in range(N):
    if j % n == 0 and j != 0:
        partes.append("|")
    partes.append(grelha[i][j])
print(*partes)

```

```

In [ ]: p_m, g_m = sudoku_resolvido(3)
        mostra(g_m, 3)

```

```

1 6 5 | 4 3 9 | 2 8 7
4 3 7 | 1 8 2 | 6 5 9
2 9 8 | 7 6 5 | 1 4 3
-----
3 5 4 | 2 1 7 | 9 6 8
6 1 2 | 3 9 8 | 5 7 4
7 8 9 | 6 5 4 | 3 1 2
-----
5 4 3 | 8 2 1 | 7 9 6
8 2 1 | 9 7 6 | 4 3 5
9 7 6 | 5 4 3 | 8 2 1

```

```

In [ ]: p_2, g_2 = sudoku_resolvido(2)
        mostra(g_2, 2)

```

```

1 3 | 4 2
2 4 | 3 1
-----
3 1 | 2 4
4 2 | 1 3

```